==========

**RAPPORT FINAL — ATELIER MACHINE LEARNING (GI2)**

**Projet : Analyse Comportementale Clientèle & Prédiction de Churn**

**Année universitaire 2025-2026**

==========

# 1. INTRODUCTION & CONTEXTE

Ce projet est un atelier pratique de Machine Learning réalisé dans le cadre du

module ML (GI2). Il simule une mission de Data Scientist au sein d'une entreprise e-commerce de cadeaux.

Objectifs métier :

- Personnaliser les stratégies marketing.
- Réduire le taux de départ des clients (churn).
- Optimiser le chiffre d'affaires.

La base de données est intentionnellement imparfaite (valeurs manquantes,

valeurs aberrantes, formats inconsistants) pour maîtriser l'ensemble de la

chaîne de traitement :

EXPLORATION → PRÉPARATION → MODÉLISATION → ÉVALUATION → DÉPLOIEMENT

# 2. ARCHITECTURE GLOBALE DU PROJET

Le projet est structuré en trois couches principales :

1. PIPELINE ML (Python / scikit-learn)

   Nettoyage, feature engineering, ACP, entraînement de classifieurs et

   tracking des expériences avec MLflow.

2. API REST (Flask)

Exposition des modèles via une API avec prédiction individuelle, batch, statistiques dataset, réentraînement et intégration MLflow.

3. FRONTEND (Vue.js 3 + Tailwind CSS)

Tableau de bord interactif pour visualiser les prédictions de churn, statistiques client et gérer l'historique.

# 3. JEU DE DONNÉES (52 FEATURES)

Fichier source :

data/raw/retail_customers_COMPLETE_CATEGORICAL.csv

Le dataset comporte 52 colonnes décrivant le comportement client :

Numériques (1–34) :

Recency, Frequency, MonetaryTotal, MonetaryAvg, MonetaryStd, TotalQuantity, CustomerTenure, Age, SatisfactionScore, Churn (binaire, target), etc.

Catégorielles (35–52) :

RFMSegment, AgeCategory, SpendingCat, CustomerType, FavoriteSeason, PreferredTime, Region, LoyaltyLevel, ChurnRisk, Gender, AccountStatus, Country, etc.

Problèmes de qualité résolus :

| Problème | Features concernées | Traitement |
|---|---|---|

| | | |
|---|---|---|
| Valeurs manquantes | Age, SupportTicketsCount, SatisfactionScore | Imputation (moyenne, médiane, mode selon la distribution) |
| Valeurs aberrantes | SupportTicketsCount, SatisfactionScore | Filtrage par seuil métier [0,7] et [0,5], remplacement par médiane/mode |
| Formats inconsistants | RegistrationDate (UK, ISO, US) | Parsing robuste multi-format avec pd.to_datetime + extraction jour/mois/année |
| Features inutiles | NewsletterSubscribed (100% constant), AvgDaysBetween Purchases | Suppression |
| IP complexe | LastLoginIP | Extraction IsPrivateIP (privée/publique) |
| Déséquilibre classes | AccountStatus, Churn | Rééquilibrage via class_weight="balanced" |

# 4. PIPELINE MACHINE LEARNING DÉTAILLÉ

Le pipeline complet est orchestré par run_pipeline.py qui exécute

5 étapes séquentielles :

---

ÉTAPE 1 — PRÉTRAITEMENT (src/preprocessing.py)

---

a) Chargement des données brutes (52 features, ~13 000 lignes).

b) Correction de l'âge (Age) :

- Skewness quasi-symétrique → imputation par la moyenne.

c) Correction SupportTicketsCount :

- Valeurs hors [0,7] remplacées par la médiane du range valide.

d) Correction SatisfactionScore :

- Valeurs hors [0,5] remplacées par le mode du range valide.

e) Parsing robuste des dates (RegistrationDate) :

- Supporte ISO (YYYY-MM-DD), UK (DD/MM/YYYY, DD/MM/YY), US (MM/DD/YYYY, MM/DD/YY), et inférence pandas.
- Extraction de RegistrationDate_day, _month, _year.
- Valeurs manquantes remplies par médiane.

f) Suppression colonnes inutiles :

- NewsletterSubscribed (100% constant)
- AvgDaysBetweenPurchases (redondante)

g) Correction AccountStatus :

- Regroupement des statuts rares : Suspended, Pending, Closed → Inactive.

h) Extraction IP (LastLoginIP → IsPrivateIP) :

- Détection IP privée/publique via le module ipaddress.

i) Encodage des variables catégorielles :

- One-Hot Encoding : Gender, AccountStatus, FavoriteSeason, PreferredTimeOfDay, WeekendPreference, CustomerType

- Label Encoding manuel ordonné :
  - LoyaltyLevel (Nouveau→0, Jeune→1, Établi→2, Ancien→3)
  - AgeCategory (Inconnu→0, 18-24→1, ... 65+→6)
  - SpendingCategory (Low→0, Medium→1, High→2, VIP→3)
  - ChurnRiskCategory (Faible→0, Moyen→1, Élevé→2, Critique→3)
  - BasketSizeCategory (Petit→0, Moyen→1, Grand→2)
  - ProductDiversity (Spécialisé→0, Modéré→1, Explorateur→2)
  - RFMSegment (Dormants→0, Potentiels→1, Fidèles→2, Champions→3)
- Mapping manuel : Region → 7 groupes géographiques encodés en dummy, Country → 46 pays numérotés.

j) Feature Engineering :

- MonetaryPerDay = MonetaryTotal / (Recency + 1)
- AvgBasketValue = MonetaryTotal / Frequency
- TenureRatio = Recency / (CustomerTenureDays + 1)
- ProductDiversityPerTrans = UniqueProducts / Frequency
- ReturnIntensity = ReturnRatio × Frequency
- Suppression des features source pour éviter la fuite de données.

Sortie : data/processed/retail_customers_processed.csv

---

ÉTAPE 2 — ANALYSE DE CORRÉLATION (src/utils.py)

---

- Génération de la matrice de corrélation de Pearson complète.
- Détection de multicolinéarité :
  - Seuil |corrélation| > 0.8
  - Calcul du VIF (Variance Inflation Factor)
  - Recommandation automatique de suppression (feature avec plus forte corrélation moyenne dans chaque paire conflictuelle).
- Visualisations exportées :

- outputs/correlation_matrix.png
- outputs/correlation_highlights.png (seuil 0.8)
- outputs/correlation_matrix_cleaned.png (après suppression)
- outputs/correlation_highlights_cleaned.png

• Suppression automatique des features redondantes avant modélisation.

---

ÉTAPE 3 — SPLIT TRAIN/TEST (src/split_data.py)

---

• Séparation 80/20 stratifiée sur la target Churn.
• random_state = 42 (reproductibilité).
• Sauvegarde CSV :
  - data/train_test/X_train.csv, X_test.csv
  - data/train_test/y_train.csv, y_test.csv

---

ÉTAPE 4 — RÉDUCTION DE DIMENSION — ACP (src/pca_transform.py)

---

• StandardScaler (fit sur train uniquement, transform sur test).
• PCA : conservation de 95% de la variance expliquée.
• Sauvegarde des transformers :
  - models/scaler.pkl
  - models/pca.pkl
  - models/pca_feature_loadings.csv (matrice des loadings)
  - models/pca_input_features.csv
• Export des données réduites :
  - data/train_test/X_train_pca.csv
  - data/train_test/X_test_pca.csv

---

ÉTAPE 5 — MODÉLISATION (src/Train_Model.py)

---

Trois algorithmes entraînés et comparés sur les données PCA :

1. Logistic Regression

   class_weight='balanced', max_iter=1000, random_state=42

2. Random Forest Classifier

   n_estimators=100, class_weight='balanced', max_depth=10,

   min_samples_split=5, random_state=42

3. Gradient Boosting Classifier

   n_estimators=100, max_depth=5, learning_rate=0.1, random_state=42

Métriques calculées pour chaque modèle :

- Accuracy, Precision, Recall, F1-Score, ROC-AUC
- Matrice de confusion (TN, FP, FN, TP)

Tracking MLflow :

- Chaque run logge : hyperparamètres, métriques, modèle sklearn.
- URI : sqlite:///mlflow.db
- Nom d'expérience : "churn_prediction"

Sélection du meilleur modèle :

- Critère : ROC-AUC maximum.
- Meilleur modèle sauvegardé localement : models/model.pkl

Résultats obtenus :

┌──────────────────────┬──────────────┐

```
| Modèle            | ROC-AUC   |
├──────────────────────┼──────────────┤
| GradientBoosting   | 0.994     |  <-- DÉPLOYÉ
| RandomForest       | 0.993     |
| LogisticRegression | 0.989     |
└──────────────────────┴──────────────┘
```

Rapport sauvegardé : reports/model_results.txt

# 5. FEATURE SELECTOR — INFÉRENCE MULTI-SORTIE (src/feature_selector.py)

Objectif : à partir de seulement 9 features connues, inférer les ~40 features restantes du profil client.

Features d'entrée (9) :

Recency, Age, Region, MonetaryTotal, Frequency, SatisfactionScore, LoyaltyLevel, CustomerType, AccountStatus

Modèle utilisé :

- MultiOutputRegressor(RandomForestRegressor(n_estimators=150, n_jobs=-1))

Pipeline :

1. Nettoyage des données brutes (mêmes fonctions que preprocessing.py).
2. Label Encoding des entrées catégorielles.
3. Label Encoding des cibles catégorielles.
4. Entraînement / évaluation (RMSE par feature cible).
5. Sérialisation de 6 artefacts :
   models/feature_selector_model.pkl
   models/feature_selector_encoders.pkl
   models/feature_selector_target_encoders.pkl

models/feature_selector_targets.pkl

models/feature_selector_cat_targets.pkl

models/feature_selector_cat_inputs.pkl

Ce modèle est utilisé par l'API pour enrichir un profil client incomplet.

# 6. API FLASK — ENDPOINTS REST (src/api.py)

Base URL : http://127.0.0.1:5000

CORS activé (origins : *)

| Endpoint | Méthode | Description |
|---|---|---|
| /api/health | GET | Santé API + URI MLflow |
| /api/inputs | GET | Métadonnées des 9 features d'entrée (min/max, options catégorielles) |
| /api/predict | POST | Prédiction churn + inférence features manquantes. Logging MLflow par prédiction |
| /api/batch | POST | Prédiction batch sur plusieurs clients. Logging MLflow agrégé. |
| /api/dataset_stats | GET | Statistiques globales (total, churn rate) |

```
├──────────────────────────────────┼─────────┼──────────────────────────────────────┤
│ /api/churn_summary               │ GET     │ Distribution churn (Loyal vs Churn)  │
├──────────────────────────────────┼─────────┼──────────────────────────────────────┤
│ /api/risk_distribution           │ GET     │ Distribution risques sur échantillon 200 │
├──────────────────────────────────┼─────────┼──────────────────────────────────────┤
│ /api/sample_predictions          │ GET     │ 10 premières lignes avec prédictions │
├──────────────────────────────────┼─────────┼──────────────────────────────────────┤
│ /api/region_churn                │ GET     │ Taux de churn par région             │
├──────────────────────────────────┼─────────┼──────────────────────────────────────┤
│ /api/train                       │ POST    │ Réentraînement des 3 modèles via l'API. │
│                                  │         │ Résultats loggés dans MLflow.        │
├──────────────────────────────────┼─────────┼──────────────────────────────────────┤
│ /api/predict_with_mlflow_model   │ POST    │ Prédiction via modèle chargé directement │
│                                  │         │ depuis MLflow (run le plus récent).  │
└──────────────────────────────────┴─────────┴──────────────────────────────────────┘
```

Gestion des modèles :

- Chargement dynamique depuis MLflow (run le plus récent).
- Fallback vers models/model.pkl si MLflow indisponible.
- Entraînement automatique du feature_selector si artefacts manquants
  (au premier démarrage de l'API).

# 7. FRONTEND VUE.JS (app_Vue/)

Stack technique :

- Vue 3 (Composition API) + Vite

- Vue Router (SPA — 3 vues)
- Tailwind CSS + PostCSS + Autoprefixer
- Lucide Vue Next (icônes)
- Chart.js + vue-chartjs (prêts à l'emploi)

Configuration proxy Vite :

- /api → http://127.0.0.1:5000

Pages développées :

1. DASHBOARD (Dashboard.vue)
   - KPIs : Total Predictions, Churn Risk, Retention, Avg Confidence
   - Quick Actions vers Predict / History
   - System Status (API, Model, Last Updated)

2. PREDICT (Predict.vue)
   - Formulaire dynamique des 9 features d'entrée.
   - Appel POST /api/predict
   - Affichage du résultat churn :
     - Probabilité de churn (%)
     - Label (Will Churn / Will Stay)
     - Niveau de risque (Low / Medium / High) avec code couleur
   - Affichage des ~40 features inférées, catégorisées :
     Monetary, Behaviour, Preferences, Profile, Other
   - Récupération dynamique des métadonnées via /api/inputs

3. HISTORY (History.vue)
   - Tableau d'historique des prédictions avec :
     - Recherche, filtre par risque, filtre par date
     - Barre de probabilité visuelle
     - Pagination

- Actions (View, Export, Delete)

- Données mockées prêtes à être remplacées par un vrai backend d'historique.

Layout (App.vue) :

- Sidebar fixe avec navigation (Dashboard, Predict, History)
- Logo "ChurnPredict" avec icône Brain
- Profil utilisateur en bas de sidebar

# 8. TRACKING & MLOPS — MLFLOW

Configuration :

- Backend : SQLite locale (mlflow.db)
- Dossier artefacts : mlruns/
- Expérience : "churn_prediction"

Utilisations :

- Tracking de chaque run d'entraînement (3 modèles × N exécutions).
- Comparaison des algorithmes sur accuracy, precision, recall, f1, roc_auc.
- Logging des prédictions API (inputs, probabilités, classes prédites).
- Chargement dynamique du meilleur modèle pour inférence.
- Réentraînement complet accessible via endpoint /api/train.

Commande UI :

```
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

# 9. SCRIPT DE PRÉDICTION AUTONOME (src/predict.py)

Fonctions disponibles :

- predict_churn(customer_data) : prédit sur un DataFrame complet (pre-PCA).

- predict_single_customer(customer_dict) : prédit sur un dictionnaire client.

Pipeline de prédiction :

Données brutes → StandardScaler → PCA → Modèle → Classe + Probabilité

Exécution directe :

python src/predict.py

(Affiche les prédictions des 5 premiers clients du jeu de test)

# 10. STRUCTURE DES DOSSIERS & FICHIERS GÉNÉRÉS

```
ML/
├── data/
|   ├── raw/                  retail_customers_COMPLETE_CATEGORICAL.csv
|   ├── processed/            retail_customers_processed.csv
|   ├── train_test/           X_train.csv, X_test.csv, y_train.csv, y_test.csv
|   |                         X_train_pca.csv, X_test_pca.csv
|   └── external/             GeoLite2-Country (données GeoIP)
├── notebooks/
|   └── nb.ipynb              (prototypage Jupyter)
├── src/
|   ├── preprocessing.py      Nettoyage + feature engineering
|   ├── utils.py              Analyse de corrélation + visualisations
|   ├── split_data.py         Séparation train/test stratifiée
|   ├── pca_transform.py      ACP + standardisation
|   ├── Train_Model.py        Entraînement 3 classifieurs + MLflow
|   ├── feature_selector.py   Modèle multi-sortie (inférence features)
|   ├── predict.py            Script de prédiction autonome
|   └── api.py                API Flask (endpoints REST + MLflow)
├── app_Vue/
```

```
│   ├── src/
│   │   ├── App.vue              Layout avec sidebar
│   │   ├── main.js              Point d'entrée Vue
│   │   ├── router/index.js      Configuration Vue Router
│   │   ├── style.css            Styles Tailwind custom
│   │   └── views/
│   │       ├── Dashboard.vue    Vue d'ensemble (KPIs)
│   │       ├── Predict.vue      Formulaire prédiction churn
│   │       └── History.vue      Historique prédictions
│   ├── package.json             Dépendances Vue
│   ├── vite.config.js           Config Vite + proxy API
│   └── tailwind.config.js       Config Tailwind
├── models/                      Modèles sérialisés (.pkl / .joblib)
│   ├── model.pkl                Meilleur modèle (GradientBoosting)
│   ├── scaler.pkl               StandardScaler
│   ├── pca.pkl                  Modèle PCA
│   ├── pca_feature_loadings.csv     Matrice des composantes
│   └── feature_selector_*.pkl       6 artefacts du feature selector
├── reports/
│   ├── model_results.txt        Comparaison ROC-AUC des modèles
│   └── report.txt               (ce rapport)
├── outputs/
│   ├── correlation_matrix.png       Matrice corrélation complète
│   ├── correlation_highlights.png   Corrélations fortes (seuil 0.8)
│   ├── correlation_matrix_cleaned.png    Après suppression
│   ├── correlation_highlights_cleaned.png  Après suppression
│   └── retail_customers_dropped.csv     Dataset après drop
├── mlflow.db                    Base SQLite MLflow
├── mlruns/                      Artefacts MLflow
├── run_pipeline.py              Orchestrateur du pipeline complet
├── requirements.txt             Dépendances Python
```

└── README.md  Documentation complète

# 11. TECHNOLOGIES UTILISÉES

Backend / Data Science :

Python 3, pandas, numpy, scikit-learn

Flask, flask-cors  → API REST

MLflow  → Tracking expériences & modèles

joblib  → Sérialisation modèles

matplotlib, seaborn  → Visualisations

geoip2, python-dotenv  → GeoIP & variables d'environnement

Frontend :

Vue 3 + Vite

Vue Router

Tailwind CSS + PostCSS + Autoprefixer

Chart.js + vue-chartjs

lucide-vue-next

# 12. INSTALLATION & UTILISATION

1. Environnement Python :

```
python -m venv venv
venv\Scripts\activate       (Windows)
pip install -r requirements.txt
```

2. Exécution du Pipeline ML :

```
python run_pipeline.py
```

Fichiers générés :

- data/processed/retail_customers_processed.csv
- data/train_test/X_train_pca.csv, X_test_pca.csv, y_train.csv, y_test.csv
- models/model.pkl, scaler.pkl, pca.pkl
- reports/model_results.txt
- outputs/correlation_*.png

3. Lancement de l'API :

```
python src/api.py
```

Accessible sur http://127.0.0.1:5000

4. Lancement du Frontend :

```
cd app_Vue
npm install
npm run dev
```

Accessible sur http://localhost:3000 (proxy API actif)

5. Visualisation MLflow :

```
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

# 13. COMPÉTENCES DÉVELOPPÉES

Au travers de ce projet, les compétences suivantes ont été mises en oeuvre :

- Exploration et nettoyage de données réelles (imputation, parsing, encodage)
- Feature Engineering (ratios, interactions, transformations)
- Analyse de corrélation et détection de multicolinéarité (VIF)
- Réduction de dimension (PCA avec conservation de variance)
- Modélisation supervisée (classification binaire)
- Évaluation multicritère (Accuracy, Precision, Recall, F1, ROC-AUC)

- Rééquilibrage de classes (class_weight balanced)
- MLOps avec MLflow (tracking, comparaison, versioning)
- Développement d'API REST avec Flask
- Déploiement d'une application full-stack (Python backend + Vue frontend)
- Sérialisation et réutilisation de pipelines de prétraitement
- Modélisation multi-sortie (RandomForest MultiOutputRegressor)

==========

FIN DU RAPPORT

==========